

PhotonWeave

Lumerical FDTD를 대체하는 GPU 가속 FDTD 워크벤치

브라우저 CAD + Python API + NVIDIA CUDA 네이티브 솔버 · MIT 라이선스 · v0.14.0.dev

The screenshot displays the PhotonWeave software interface, which is a web-based CAD and simulation environment. The top bar shows the application name "PhotonWeave" and the current simulation type "FDTD" with a "2D TMz" waveguide structure. The interface is divided into several panels:

- Top Bar:** Includes a menu (File, Edit, View, Materials, Simulation, Feature checklist, Flux results, Help) and a status bar indicating "CPU solver" and "v0.14.0.dev".
- Design Panel:** Contains a toolbar with icons for opening, saving, and inspecting files, as well as tools for creating various geometric shapes (Rectangle, Circle, Ring, Sphere, Polygon) and simulation objects (FDTD region, Dipole, Sheet source, TFSF box, Time monitor, DFT / Flux). It also includes buttons for duplicating, deleting, and fitting the view.
- Objects Tree:** A hierarchical list of objects in the simulation, including "model", "FDTD", "STRUCTURES", "waveguide", "SOURCES", "source", "MONITORS", "input", and "output".
- Simulation Region:** A 3D perspective view of the simulation setup, showing a blue waveguide structure within a yellow rectangular region.
- Object Properties:** A panel on the right that allows users to configure the simulation parameters. It includes sections for "General" (dimension, resource, precision, CUDA kernel, frequency monitor kernel), "Geometry" (x span, y span, z span), and "Mesh settings" (mesh type, independent axis spacing, dx = dy = dz).
- Results View:** A panel at the bottom left that provides a summary of the simulation results, including a "Simulation log" and a "Python script" section.

The main workspace is divided into four quadrants, each showing a different view of the simulation: "XY plane", "Perspective", "XZ plane", and "YZ plane". Each view includes a coordinate system and a "Snap" button. The "XY plane" view shows a 2D cross-section of the waveguide with a red dot indicating the source position. The "XZ plane" view shows a 2D cross-section of the waveguide with a red dot indicating the source position. The "YZ plane" view shows a 2D cross-section of the waveguide with a red dot indicating the source position. The "Perspective" view shows a 3D view of the waveguide structure.

6.5 – 15.2×

Lumerical CPU(16 threads) 대비 기록된 wall time 비

과거 빌드 · 동

일정확도 아님

9.6 – 17.1×

동일 격자·소스·모니터에서
오픈소스 GPU 기준선 대비

≈ 40×

같은 4케이스 배치:

NumPy CPU 47.04

s → CUDA 1.175 s

46.7 / 85.9×

adjoint 기울기 1회:

CPU 대비 GPU resident

(FP32 / complex FP64)

54 GiB

48 GB VRAM을 넘는

상태를 streaming으로

전진+역전파 완료

4.4e-6

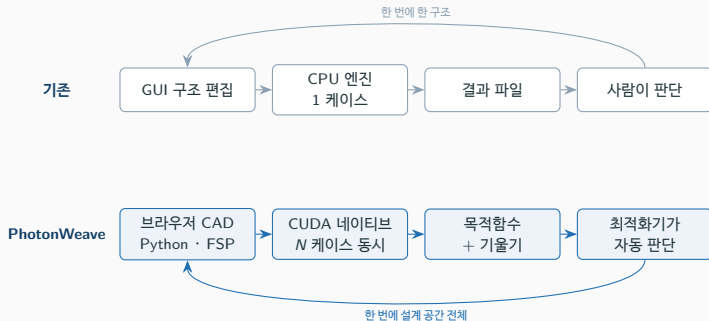
박막 해석해 대비 $R+T-1$

잔차 (오늘 CPU에서 재현)

순서: 왜 만들었나 → 무엇을 만들었나 → 정확도 → 속도 → 역설계 → 한계

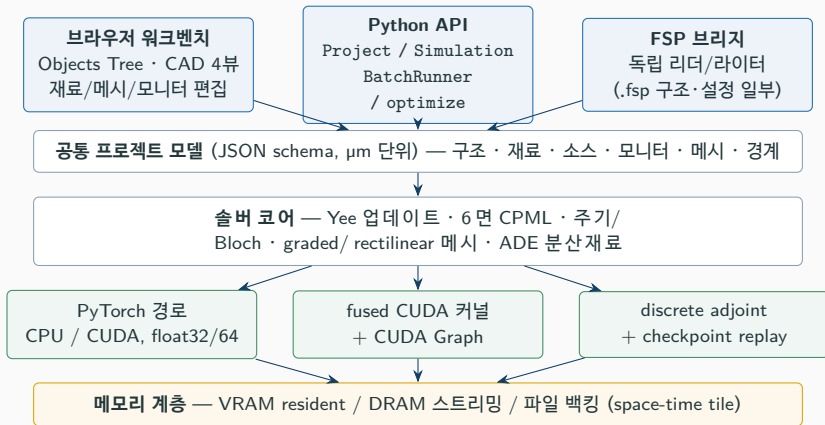
왜 직접 만들었나

- 스윙/역설계는 **케이스 수만큼 순차 실행**, 동시 실행 개수는 보유 라이선스에 묶임. 우리가 비교 기록을 가진 구성은 **CPU 16 스레드 엔진**입니다 (상용 GPU 엔진은 미비교)
- 목적함수의 **기울기**를 사용자 코드에서 받을 수 없어, 설계 루프가 사람의 판단 속도에 묶임



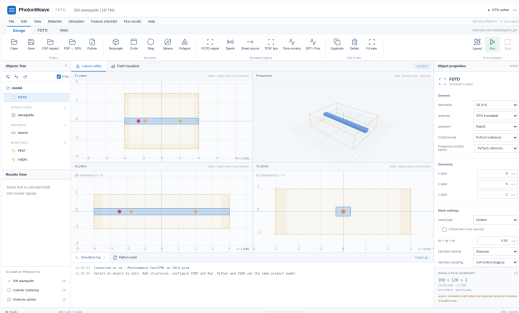
바꾼 것: 익숙한 UI는 유지, 계산은 **GPU 배치**로, 기울기는 **discrete adjoint**로 노출.

시스템 구조

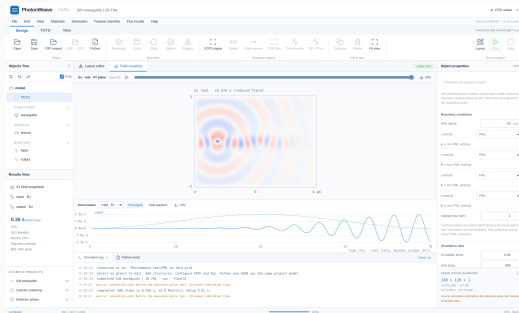


그리드/백엔드 기반은 오픈소스 flaport/fdtd(MIT)를 사용하고, Yee 미분 · CPML · 주기/Bloch · CUDA Graph 실행은 직접 구현했습니다.

익숙한 워크플로: Layout → Run → Analysis



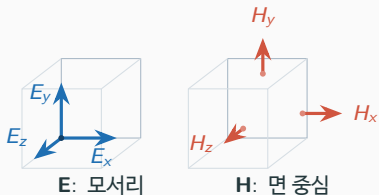
구조 트리 · CAD 4뷰 · 객체 속성 · 재료/메시 설정



필드 스냅샷 애니메이션 · 모니터 시계열/스펙트럼 ·
NPZ/CSV · Python export

브라우저에서 만든 구조가 그대로 Python 객체입니다. UI → JSON → 스크립트가 같은 모델.

엔진: Yee/CPML을 직접 구현하고 CUDA로 융합



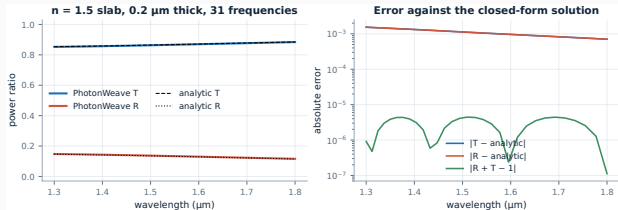
3D 6성분 + 2D TMz/TEz, float32/64,
실수장과 복소장 (Bloch), CFL 기본 0.99.

- **경계:** 면별 독립 CPML(3D 6면) + 주기/Bloch 쌍
대칭/PEC/PMC는 미구현
- **소스:** 점 dipole, soft sheet, one-way 주기 평면,
closed TFSF box (수직 입사)
- **메시:** uniform / graded / 축별 독립 rectilinear,
직사각형 CFL
- **재료:** 상수 n + 수동 Drude/Lorentz 최대 16 극점,
측정 n/k 피팅
- **실행:** fused Yee/CPML CUDA 커널 + CUDA
Graph + **케이스 배치 축**

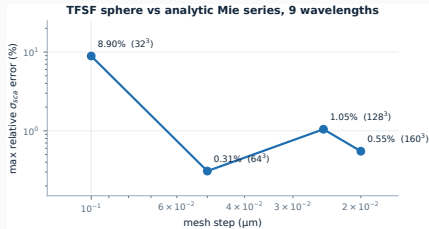
배치 축이 핵심

독립 구조 N 개를 한 번의 커널 launch로 전진시키고,
결과는 개별 실행과 **bitwise 동일**합니다.

정확도: 해석해와 비교

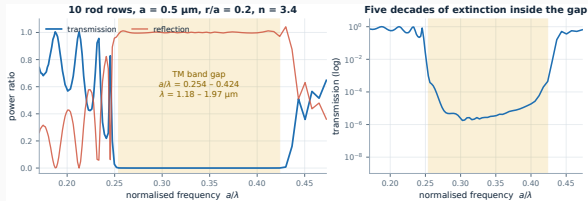


$n = 1.5$ 박막: 최대 오차 $|T - T_{\text{exact}}| = 1.53\text{e-}3$, 에너지 잔차 $4.44\text{e-}6$. 오늘 CPU float64로 1.4초 만에 재현.

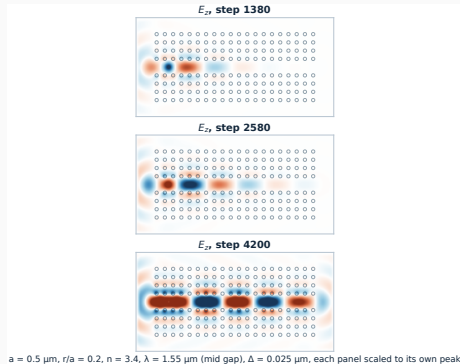


TFSF 구 산란 대 Mie 급수. 0.05 μm에서 0.31%지만 오차가 단조 감소하지 않습니다 — 정량 결과에는 수렴 테스트 필요.

우리 연구 대상으로 직접 계산: 밴드갭과 W1 도파로

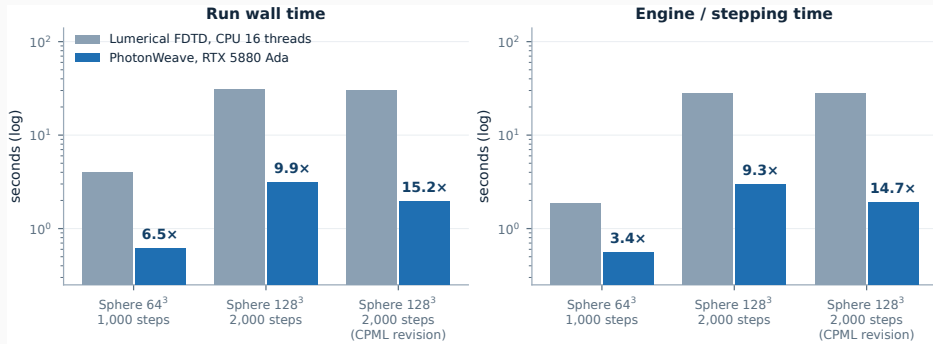


정사각 rod 격자 ($a = 0.5 \mu\text{m}$, $r/a = 0.2$, $n = 3.4$) 10열,
broadband one-way 평면 소스 + 주기 경계 + flux 모니터.
CPU에서 약 30초.



같은 결정의 W1 선결함을 밴드갭 안쪽 $1.55 \mu\text{m}$
로 여기. 결정 안쪽은 evanescent, 채널을 따라
전파. 160 rod / 6000 스텝 / CPU 22초.

속도 1: Lumerical FDTD와의 기록된 비교

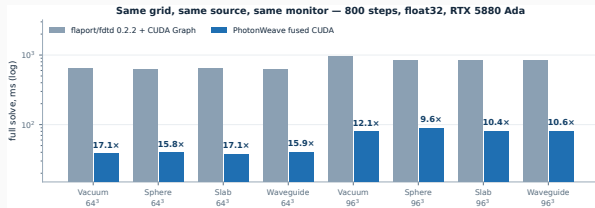


6 μm 도메인의 $n = 2$ 구, 1.55 μm Gaussian dipole, 포인트 모니터 1개, 3회 측정의 중앙값.

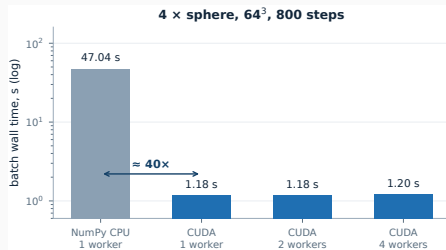
이 표를 읽는 법

과거 빌드의 기록이며 현재 릴리스의 검증된 speedup이 아닙니다. dipole 정규화·필드 보간·흡수 경계가 달라 동일 정확도 비교가 아니고, Lumerical GPU 엔진과는 측정하지 않았습니다.

속도 2: 같은 물리 설정에서 측정한 것



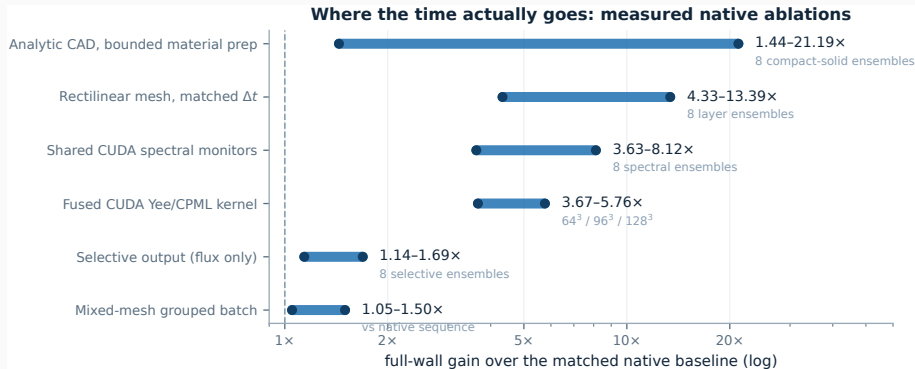
두 엔진에 동일한 voxel 유전율·시간 스텝·소스·모니터를 주고, 공개 코드에 CUDA Graph 어댑터까지 붙여 준 비교. 포인트 트레이스 상대 L2 <0.04%.



4개 독립 구 케이스, 64³, 800 스텝. 워커를 2/4개로 늘려도 이 문제에서는 더 빨라지지 않습니다. 배치 축은 32³에서 **112**

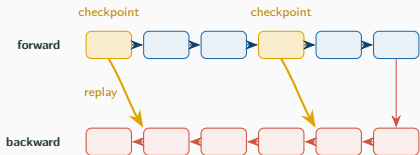
cases/s까지 오르지만, 64³의 8/16 케이스에서는 **오히려 느려집니다** (부록).

속도 3: 시간이 실제로 줄어든 곳

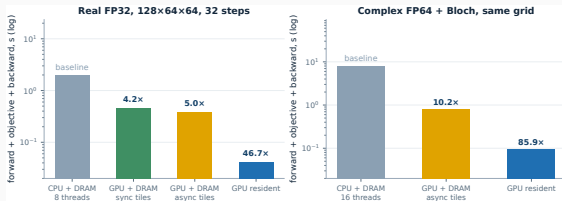


단일 커널 최적화만이 아니라 **호스트 준비 시간 · 불필요한 셀 · 저장하지 않아도 되는 출력**을 제거한 것이 워크플로 시간에 더 크게 기여했습니다. 모든 행은 동일 조건의 네이티브 baseline과 비교한 full-wall 시간입니다.

역설계: discrete adjoint로 기울기를 직접 계산

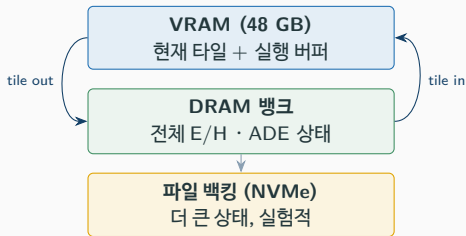


- 파라미터가 P 개여도 기울기 1회 = forward 1회 + backward 1회 (유한차분은 $P+1$ 회)
- 전체 시간 이력 대신 **checkpoint replay**로 메모리 제한
- 대각 ε , Torch geometry, Drude/Lorentz 파라미터까지 미분

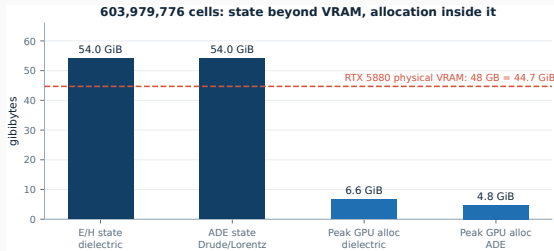


같은 코드 경로·같은 격자(128×64×64, 32 스텝)의 forward + 목적함수 + backward 전체 시간. 모든 모드가 신호·기울기 검사를 통과, CPU는 최적 스레드 수 사용. 이 문제는 VRAM 안에 들어갑니다.

VRAM보다 큰 문제도 푼다



space-time tile: 한 칸 Yee 의존 cone을
halo로 포함해 손실 없이 분할



603,979,776 셀, E/H 54 GiB 상태에서 **10 스텝 전진 + 1차 기울기 검증 완료** (peak GPU 할당 6.6 GiB, 52분).
용량 증명이지 속도 우위가 아닙니다. 실제 응용
지속시간에서의 성능은 미검증.

아직 안 되는 것과 다음 단계

한계

- 이득 매질 · 비선형 · 이방성 미지원, 대칭/PEC/PMC 미구현
- 경사 입사 one-way/TFSF, 도파로 모드 소스, far-field, mode port 미구현
- 체크리스트 1,661행 중 구현 160 / 부분 231 (속성 수, 완성도 비율 아님)
- 상용 솔버와 **동일 정확도 조건의 재측정**이 아직 남아 있음

다음 단계

- 공통 정확도 목표로 상용 솔버 재측정, Lumerical GPU 엔진 포함
- mode/port 정규화 목적함수 → 실제 역설계 검증
- 통합 메모리 정책 (resident/streamed 자동 선택)
- 랩 적용: PhC/metasurface 스왑을 배치로 이전, GPU 워크스테이션 공유

필요한 구조·조건을 알려주시면 그 케이스로 먼저 벤치마크하고, 정확도 게이트를 붙여서 가져오겠습니다.

요약

상용 FDTD와 같은 작업 순서를 유지한 브라우저 + Python
워크벤치

같은 물리 설정에서 우리 CPU 경로와 공개 GPU 기준선보다 빠른
CUDA 엔진

설계 루프를 위한 배치 실행과 discrete adjoint, VRAM을 넘는
메모리 계층

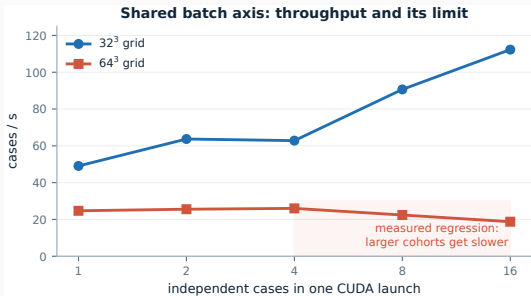
속도 수치는 조건과 한계를 함께 기록 — 상용 솔버 동일정확도
비교는 진행 중

부록: 그래서 지금 다시 해야 하는 비교

필요한 비교	현재 상태
현재 PhotonWeave vs Lumerical CPU, 공통 정확도 목표	새 matched validation 필요
현재 PhotonWeave vs Lumerical GPU, 같은 RTX 5880	미측정
다구조 배치 / 역설계 throughput vs Lumerical	미측정
오픈소스 GPU 기준선 (flaport/fdtd + CUDA Graph) 대비	측정 완료
네이티브 CPU vs GPU, 동일 코드 경로 해석해 (박막, Mie) 대비 정확도	측정 완료 측정 완료, 한계 명시

랩미팅에서 주장하는 범위: "같은 물리 설정에서 우리 GPU 경로가 우리 CPU 경로와 공개 GPU 기준선보다 빠르다".

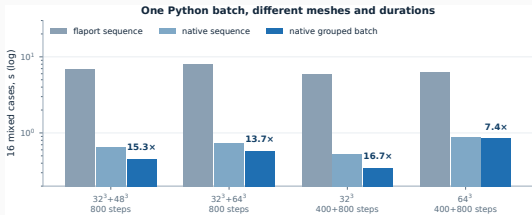
부록: 배치 축의 한계



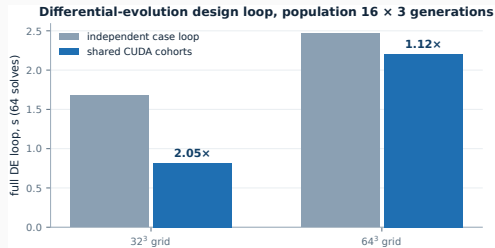
- 32³에서는 16 케이스까지 throughput 증가 (**112 cases/s**)
- 64³에서는 8, 16 케이스에서 **오히려 느려짐**
- cohort_size로 한 번에 묶는 케이스 수를 제한
- tune_tensor_batch()가 측정으로 크기를 고르지만 튜닝 자체가 비용 (64³에서 회수에 144~420 양상블)

출처: README.md · ensembles.json

부록: 혼합 메시 배치와 설계 루프

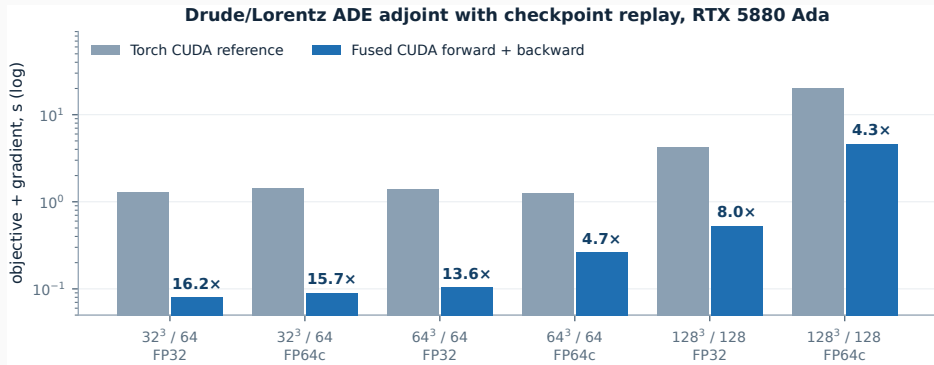


메시와 지속시간이 섞인 16 케이스를 한 Python 배치로, 호환 케이스를 자동 그룹화하고 입력 순서로 결과를 복원합니다.



블랙박스(미분 없는) differential evolution 루프, population 16 × 3세대 = 64회 forward.

부록: 분산 재료 (Drude/Lorentz) adjoint



ADE의 P/Q 상태까지 checkpoint replay에 포함하고, 재료 기울기는 atomics 없이 블록 리덕션으로 모읍니다. 공유 파라미터는 compact하게 유지되어 극점 수가 늘어도 메모리가 선형 증가하지 않습니다. 같은 연산(전진 + 목적함수 + 역전파)을 우리 Torch 경로와 비교한 내부 측정입니다.

부록: 설계 루프 코드

```
import torch
from photonweave import (DifferentiableSimulation,
                          AdjointOptions, smooth_sphere_epsilon)

model = DifferentiableSimulation(project,
                                AdjointOptions(checkpoints=4, storage="host"))
radius = torch.nn.Parameter(
    torch.tensor(0.25, dtype=torch.float64))
opt = torch.optim.Adam([radius], lr=0.003)

for step in range(iterations):
    opt.zero_grad()
    epsilon = smooth_sphere_epsilon(
        project.region, radius, width=0.09, inside=3.0)
    result = model(epsilon) # forward
    loss = result.signals[:, 0].abs().square().mean()
    loss.backward() # discrete adjoint
    opt.step()
```

examples/differentiable_design.py의
실제 코드입니다. 스트리밍 실행은
StreamedSimulation이 같은 자리에
들어갑니다. 포트 정규화된 모드 목적함수와
응용 수준 역설계 검증은 진행 중입니다.

부록: 기존 .fsp 자산과 오픈소스 지형

FSP 브리지

- 벤더 런타임 없이 독립 파싱: box, 회전 타원체/원기둥, 부분 링, polygon extrusion
- 메시 간격 · span · CAD/PML 경계 · CFL, 소스 대역/위상, 모니터 스펙트럼 writeback
- 편집하지 않은 바이트는 보존

지원하지 않는 물리는 변환을 막습니다. 외부 도구의 새 레코드 수용 · 그룹 · 일반 호환성은 미해결.

오픈소스 GPU FDTD 대비

- **FDTDx (JAX):** 역설계는 **우리보다 앞섬**
- **fdtdz (JAX + 전용 CUDA):** 매우 빠른 전용 범위, z 크기 제약, 등록된 VJP 없음
- **flaport/fdtd:** 우리가 기반으로 사용 · 출처 표기
- **fdtd3d (C++/MPI):** **단일 격자 MPI는 우리에게 없음**

같은 GPU에서 속도를 측정하지 못했습니다 (Linux CUDA 환경 부재). 기능 차이는 속도 우위가 아닙니다.

부록: 수치 규약과 측정 환경

- 길이·파장은 μm , API 시간 배열은 초
- 진폭은 reduced field (보정된 V/m 아님)
- E와 H는 시공간에서 엇갈려 있어 곧바로 곱해 Poynting으로 쓰지 않음
- 평면 모니터: 6성분 복소장 + 부호 있는 flux, reference 정규화 필요
- 재료: 상수 $n \geq 1$, 수동 등방 Drude/Lorentz 최대 16 극점
- 모든 벤치마크는 3회 반복 중앙값, warmup 후 측정, cold compile 제외 (3회는 신뢰구간을 주지 않음)

GPU 측정	RTX 5880 Ada 48 GB, PyTorch 2.10
보조 GPU	RTX 3060 (복소 FP64 측정)
CPU 비교	Xeon w3-2435, i7-12700
외부 기준선	flaport/fdtd 0.2.2 + CUDA Graph 어댑터
상용 기록	Lumerical CPU 1 프로세스 16 스레드 API 버전 문자열 8.31.3633 (v241)
정확도 게이트	트레이스 1%, flux 2%, 복소 DFT 1% 네이티브 간 bitwise

Lumerical 계산 결과·필드·프로젝트 파일은 저장소에 포함하지 않으며 타이밍 요약만 기록했습니다. 해당 표는 공개 배포 전 라이선스 검토 대상입니다.